

SOFTWARE DESIGN SPECIFICATION (SDS)

Smart Expense Tracker Web System



Prepared by: M.C. Nuwan

Student ID: RAT/IT/2324/F/0055

Department: Department of Information Technology

Institution: Advanced Technological Institute (ATI), Rathnapura

Course Module: HNDIT 4052 - Final Year Project

Supervised by: Mr. Thilina Bandara

Contents

1. Introduction & Architectural Overview.....	1
1.1 Purpose.....	1
1.2 Architectural Pattern	1
2. System Functionality & Behavioral Design	2
2.1 Use Case Diagram Analysis & Behavioral Boundaries	2
3. Data Design & Relational Schema	4
3.1 Entity-Relationship (ER) Diagram Description	4
4. User Interface & Wireframe Design Architecture	6
4.1 Low-Fidelity Wireframes Blueprint Decomposition	6
4.2 High-Fidelity User Interface Theme & Presentation Layouts	7
4.3 Screen-by-Screen Functional Mapping	12
4.4 Budget Management & Analytical Reports Wireframe	12

1. Introduction & Architectural Overview

1.1 Purpose

This Software Design Specification (SDS) translates the complete set of functional and non-functional requirements defined in the Smart Expense Tracker Web System Software Requirements Specification (SRS) into a highly rigorous, technical, and structural blueprint for final physical software implementation. The absolute core intent is to bridge the gap between abstract user requirements and actual codebase constraints, acting as an authoritative technical reference manual across the entire engineering lifecycle.

The operational scope of the Smart Expense Tracker Web System encompasses the complete lifecycle of personal financial tracking. This includes dynamic multi-source income categorization, granular expense tracking, custom lifecycle parameter category definitions, real-time threshold budgeting rules, asynchronous statistical analytics dashboard rendering, and robust system auditing trails.

1.2 Architectural Pattern

The Smart Expense Tracker Web System strictly utilizes a decoupled 3-Tier Client-Server Architectural Pattern, structurally implemented using the standard MERN (MongoDB, Express.js, React.js, Node.js) full-stack JavaScript application environment. This architecture establishes a strict separation of concerns, high maintainability, stateless backend operations, and horizontal scalability across all transactional layers.

- **Presentation Layer (Frontend Component):** Constructed as a modular React.js Single Page Application (SPA). It processes client state management via Context APIs/Hooks, captures input vectors, intercepts user interactions, and dynamically updates the browser viewports without triggering hard refreshing cycles.
- **Application Layer (Backend Server Component):** Built as an Express.js RESTful web service running inside a Node.js engine environment. This layer establishes standard routing frameworks, implements cryptographic session confirmation mechanisms, processes math-intensive budget aggregation logic, and transforms errors into uniform JSON payloads.
- **Data Layer (Storage Engine Component):** Powered by a non-relational, document-oriented MongoDB repository. It securely stores business records as high-performance, flexible BSON document collections managed via custom Mongoose Object Data Modeling (ODM) definitions.

2. System Functionality & Behavioral Design

2.1 Use Case Diagram Analysis & Behavioral Boundaries

The system's behavioral framework and interaction boundaries are visually structured around an authentication barrier that separates public operations from restricted financial workflows. The diagram maps out three distinct actor classes and their access path limitations:

- **Guest User (Unauthenticated Visitor):** Positioned outside the core system boundary. The diagram depicts this actor with limited lines of interaction, restricted entirely to the registration and login use cases.
- **Authenticated System User:** Positioned inside the secure operational boundary. The diagram illustrates this actor as the primary initiator for the core functional use cases, including Income CRUD operations, Expense CRUD operations, and Custom Category configurations.
- **System Administrator:** Mapped with specialized oversight lines. This actor interacts with administrative use cases, including system logs viewing and data backup maintenance.
- **The Authentication Wall:** Graphically represented as a gatekeeping threshold. It illustrates that moving from a Guest state to an Authenticated state requires a verified credential validation token, which unlocks the inner use case modules.

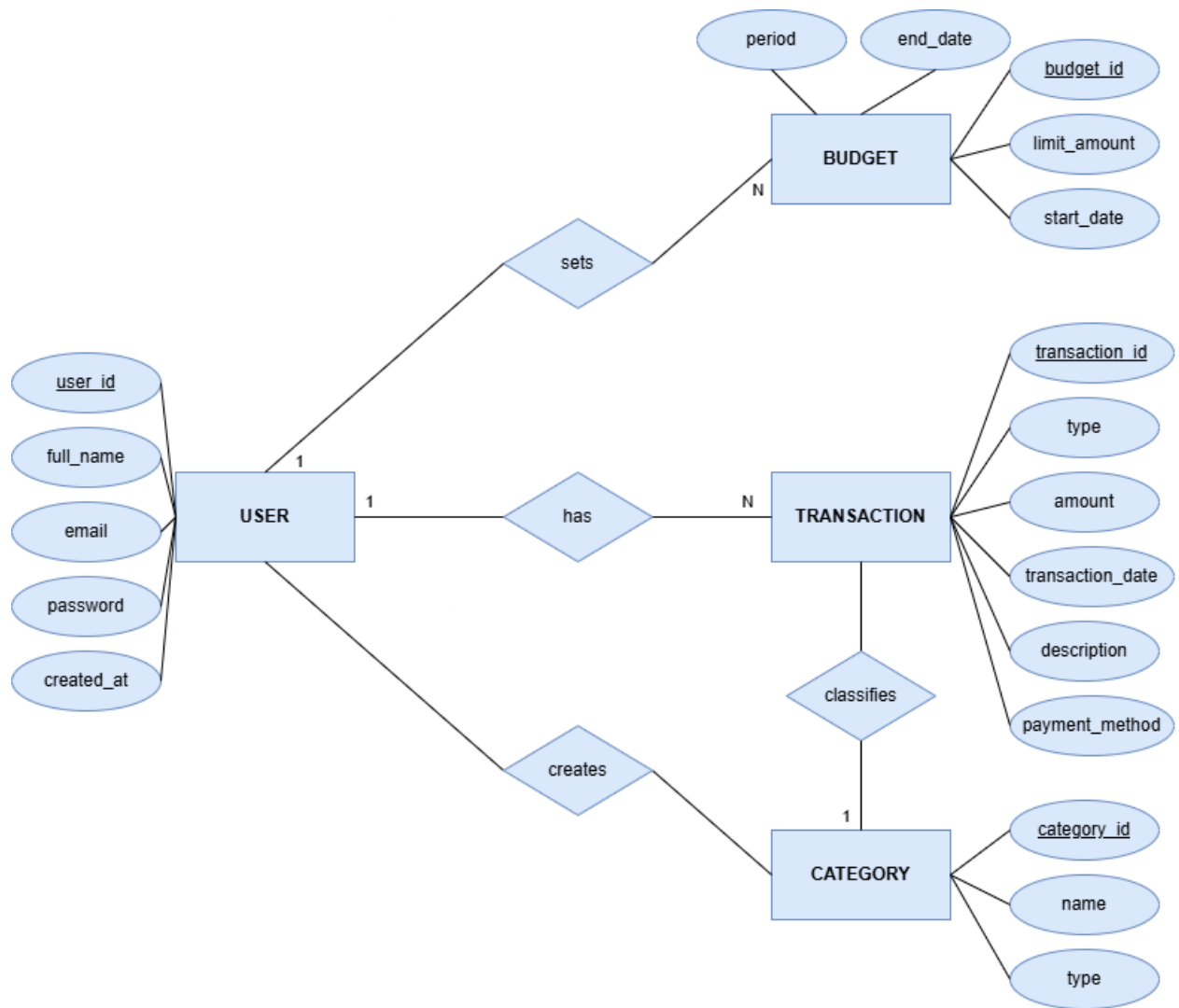


3. Data Design & Relational Schema

3.1 Entity-Relationship (ER) Diagram Description

The Entity-Relationship diagram maps out the logical structure and structural data nodes of the system's storage tier. It visually defines five core entities organized to maintain clean data tracking without duplication:

- **Users Entity (The Root Node):** Depicted as the primary parent table. It holds core profile attributes such as Name, Email, and hashed credentials.
- **Incomes & Expenses Entities:** Configured as dependent transactional data buckets. They display attributes for Amount, Timestamp/Date, and Description.
- **Categories Entity:** Arranged as a shared lookup block. It connects to both income and expense streams to provide classification names and visual color identifiers.
- **Budgets Entity:** Positioned as an analytical evaluation node. It contains attributes tracking spending limits and current month-to-date totals.
- **Cardinality and Structural Connections:** The diagram illustrates a strict **1-to-Many (\$1:N\$) relationship** branching outwards from the central Users entity to all other transactional records. This signifies that an individual user account owns multiple unique transaction entries, category groups, and budget allocations, while each record traces back to exactly one specific user footprint.

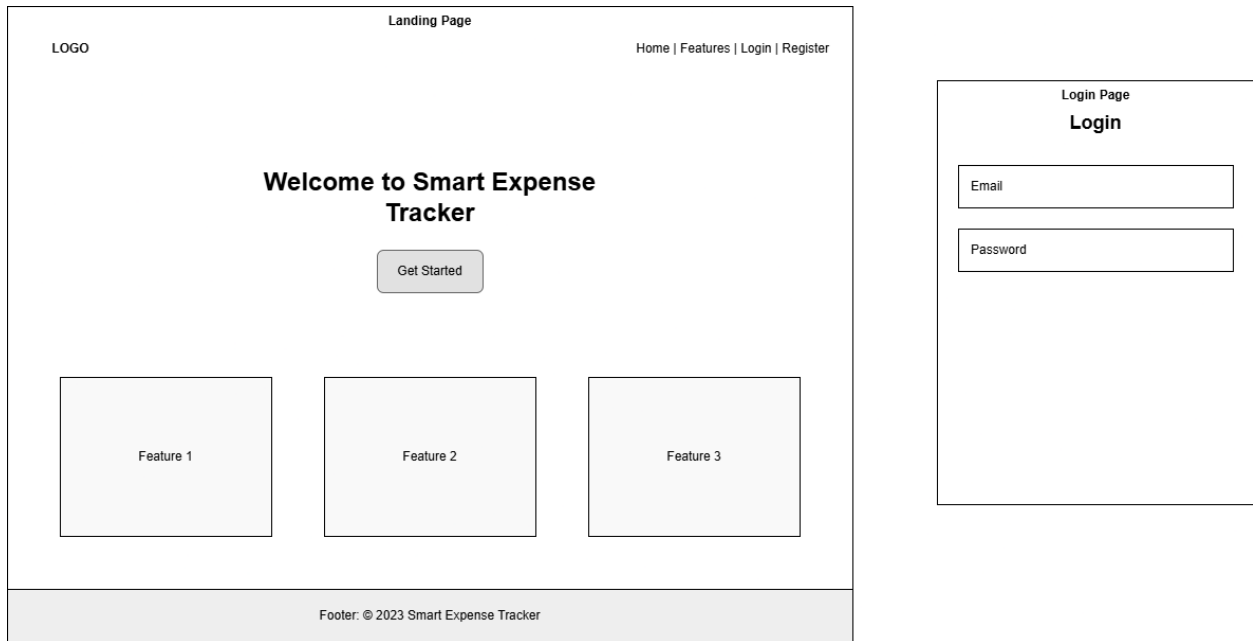


4. User Interface & Wireframe Design Architecture

4.1 Low-Fidelity Wireframes Blueprint Decomposition

This structural flow diagram models the component paths and sequential loops that occur across the application layers during a standard operation lifecycle:

- **The Request Initiation Path:** Illustrated as a forward-moving vector starting at the Client Browser interface. It depicts user actions passing through security verification middleware checking for valid session signatures.
- **The Application Routing Component:** Depicted as the central controller hub. The diagram shows incoming validation traffic being sorted and sent to the correct functional backend router based on the requested transaction type.
- **The Data Round-Trip Cycle:** Visualized as a data mapping loop between the backend router schemas and the MongoDB database collections. It shows information being pulled, formatted, and pushed back up through the layers to update the interface view presentation without a complete page refresh.



4.2 High-Fidelity User Interface Theme & Presentation Layouts

The dashboard wireframe layout maps out the main interface that appears immediately after a user successfully logs in. It is organized into three major functional view zones:

- **Control Navigation Sidebar:** Positioned along the left vertical boundary. It visually groups navigation access buttons to let users jump between the Overview Dashboard, Ledger sheets, Budget settings, and Account preferences.
- **Summary Scorecards:** Positioned as three distinct metric blocks across the top horizon of the canvas. They are arranged as independent, high-contrast containers to immediately highlight current monthly totals for Income, Outbound Expenses, and Net Balance.
- **Trend Analysis Canvas:** Occupies the central viewport area. It is a large visualization module dedicated to rendering dynamic historical trend charts that compare financial performance across custom time periods.



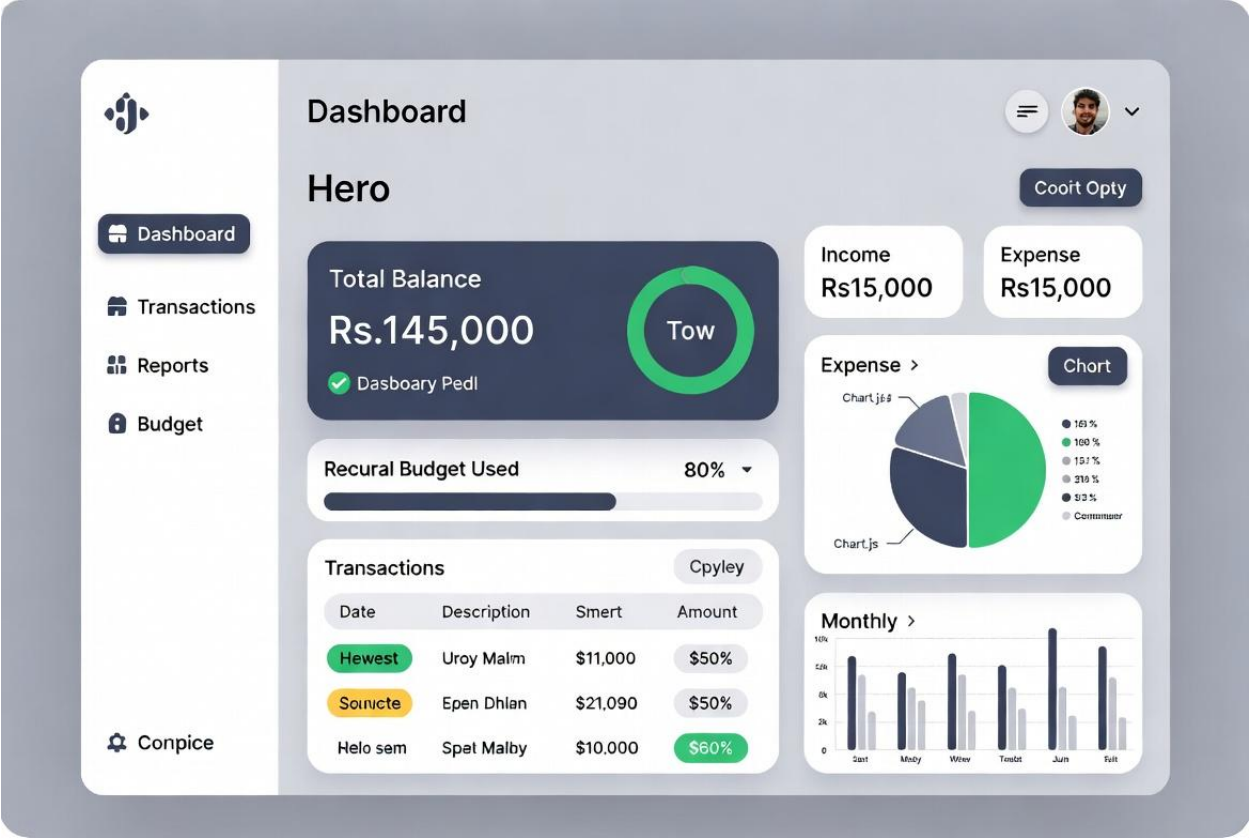
Smart Expense Tracker

Email

Password

Login

Register



Expense

Income

Expense

Amount

| 700

Date

11



Category

Description | 3190



Myltohet

Payment method



Save Transaction

🔍 Search transactions...

All

Income

Expense

01 Jan 2017 - 31 Jan 2017

Date	Description	Category	Amount	Actions
2017-01-19	Sean Transaction	Income	LKR 245,250	<button>Edit</button>
2017-01-13	Description	Expense	LKR 32,700	<button>Delete</button>
2017-01-13	Qutv Transaction	Expense	LKR 32,700	<button>Delete</button>
2017-01-14	Qutv transaction	Expense Aituvil	LKR 32,700	<button>Delete</button>
2017-01-15	Qutv transaction	Experition	LKR 81,750	<button>Delete</button>
2017-01-16	Qutv transaction	Lroups	LKR 58,860	<button>Delete</button>
2017-01-16	Qutv transaction	Replayer ₹ituvil	LKR 52,860	<button>Delete</button>
2017-01-17	Qutv transaction	Replayer ₹ituvil	LKR 32,700	<button>Delete</button>
2011-01-18	Qutv transaction	Epplayer Muow	LKR 62,130	<button>Delete</button>

↑<1-6 of 6<button>

Export

4.3 Screen-by-Screen Functional Mapping

This layout depicts the workspace dedicated to recording modifications and tracking user transaction logs. The wireframe splits data collection and viewing into two main modules:

- **Structured Form Entry Module:** Placed in the upper section of the layout. It maps out user input boxes designed for clean data capture, displaying designated fields for numerical values, category drop-down selectors, calendar date-pickers, and comment inputs.
- **Historical Grid Listing:** Positioned underneath the entry form. It displays data fields in a tabular sheet format, arranging transaction records chronologically. The right-hand column of the table includes explicit button markers for item editing and data deletion.

4.4 Budget Management & Analytical Reports Wireframe

Category Threshold Progress Meters: Configured as a vertical stack of horizontal bar components. Each bar illustrates an active category limit, showing current expenditures filled inline against a maximum spending threshold marker. This setup provides an immediate visual warning if an expense category approaches its budget limit.

Report Extraction Panel: Positioned at the base of the interface layout. It provides a grouping of data-filtering components (date-range boundary parameters) alongside download button markers for generating clean document exports in PDF or CSV format.